

CS305 SDN Project Report

Tong Wu (12412460)

Zhikai Wang (12410615)

Yunzang Li (12410922)

June 4, 2026

1 Introduction

This project implements a centralized Software-Defined Networking controller using os-ken and Mininet. The network is modeled as a single-subnet Layer-2 environment with hosts and OpenFlow switches. Our controller supports three required functions:

- dynamic IP allocation through DHCP,
- shortest-path forwarding based on the current topology,
- traffic filtering through a switch-level firewall.

In addition to the required tasks, we also implemented bonus functionality and experiments, including DNS support, DHCP lease improvements, TCP congestion experiments, and a bufferbloat study.

2 Architecture

The system is built around a centralized controller that maintains the global topology state and installs OpenFlow rules into switches. Mininet creates the virtual hosts, switches, and links. Hosts generate traffic, while the controller decides how the network should react.

The architecture can be described in four layers:

- **Topology and switching layer:** maintains switch, link, and host state; computes shortest paths; installs forwarding rules.
- **DHCP layer:** processes DHCP messages and manages IP allocation, pending offers, leases, and renewals.
- **Firewall layer:** parses deny rules from the JSON rule file and installs high-priority drop flows.
- **Bonus DNS layer:** answers DNS queries according to static DNS records.

2.1 Core Files

- `controller.py`: main controller logic. It receives switch, host, link, port, and packet events; maintains the topology view; handles ARP; delegates DHCP and DNS processing; and installs forwarding and firewall rules.
- `dhcp.py`: DHCP server implementation, including allocation, lease management, timeout reclaim, and renewal support.
- `firewall.py`: firewall rule parsing and installation logic.

- `dns_server.py`: bonus DNS implementation.
- `ofctl_utilis.py`: OpenFlow helper utilities for flow installation, packet-out operations, and related switch interaction.
- `tests/`: Mininet-based validation scripts for required and bonus parts.

2.2 Control Logic

The controller maintains the following shared state:

- online datapaths and OpenFlow control abstractions,
- learned hosts and IP-to-MAC mappings,
- switch adjacency information,
- inter-switch output port mappings,
- down-port information used for topology refresh.

The forwarding logic is built on top of this global view. Once host locations and switch links are known, the controller computes shortest paths and installs destination-MAC-based flow rules on the relevant switches.

2.3 Representative Code Snippets

The following simplified snippets illustrate the core implementation ideas.

DHCP address reservation and reclaim.

```
def allocate_ip(client_mac):
    reclaim_expired_leases()
    if client_mac in ip_pool:
        return ip_pool[client_mac]
    if client_mac in pending_offers:
        return pending_offers[client_mac]["ip"]
    for ip in build_ip_range():
        if ip not in in_use_ips() and ip not in bad_ips:
            pending_offers[client_mac] = {
                "ip": ip,
                "timestamp": now(),
            }
            return ip
    return None
```

Shortest-path forwarding refresh.

```
for src_mac, src_info in hosts.items():
    for dst_mac, dst_info in hosts.items():
        if src_mac == dst_mac:
            continue
        path = shortest_path(src_info["dpid"], dst_info["dpid"])
        for dpid in path:
            out_port = next_hop_port(dpid, path, dst_info["port"])
            install_dst_mac_flow(dpid, dst_mac, out_port)
```

Firewall deny-rule installation.

```
for dpid, ofctl in ofctls.items():
    for rule in rules:
        if rule.action != "deny":
            continue
        ofctl.set_flow(
            priority=60000,
            dl_type=IP,
            nw_src=rule.src_ip or 0,
            nw_dst=rule.dst_ip or 0,
            nw_proto=proto_to_number(rule.proto),
            tp_src=normalize_port(rule.src_port),
            tp_dst=normalize_port(rule.dst_port),
            actions=[],
        )
```

3 Implementation Details

3.1 DHCP

The DHCP module implements the standard DISCOVER/OFFER/REQUEST/ACK workflow. Hosts are created without valid initial IP addresses in the test scripts, so IP allocation must be handled by the controller.

The implementation in `dhcp.py` keeps track of:

- pending offers,
- confirmed leases,
- lease expiration timestamps,
- rejected addresses that should not be reused.

We also extended the baseline DHCP design with practical improvements:

- configurable address pool and netmask,
- offer timeout reclaim,
- lease expiration and renewal support,
- conflict-free allocation under concurrent requests.

3.2 Topology Learning and Shortest-Path Switching

The switching part is implemented in `controller.py`. The controller:

- learns hosts through gratuitous ARP and ARP requests,
- learns switch-to-switch links through LLDP and topology APIs,
- updates internal topology state on switch, host, link, and port events,
- computes shortest paths on the switch graph using breadth-first search,
- installs destination-MAC forwarding rules on the switches.

This design supports both the basic triangle topology and complex topologies with loops and dynamic changes.

3.3 Firewall

The firewall module reads policy rules from `firewall_rule.json`. Each rule can match source IP, destination IP, transport protocol, source port, and destination port. The module converts each deny rule into a high-priority OpenFlow drop flow.

The firewall rules are installed on every switch through `Firewall.install_rules`. Since the firewall rules have higher priority than normal forwarding rules, they override shortest-path forwarding whenever traffic matches a deny rule.

3.4 Bonus Components

DNS. The DNS bonus extends the controller with a controller-hosted DNS service. DNS requests are detected in `controller.py` and answered according to static records defined in `dns_records.json`.

DHCP Lease Management and RFC Optimization. The DHCP bonus extends the baseline allocator with practical lease lifecycle management and stronger RFC 2131 compliance. The implementation supports lease expiration reclaim, OFFER timeout cleanup, lease renewal, atomic reservation to avoid duplicate allocation, and conflict handling through DHCP DECLINE.

TCP Congestion. The TCP congestion bonus is an experiment built with Mininet rather than a controller-side feature. It compares TCP algorithms such as Reno and Cubic under the same bottleneck and reports throughput and fairness.

Bufferbloat. The bufferbloat bonus is another Mininet experiment. It studies the tradeoff between throughput and queueing delay under different queue sizes.

4 Team Division of Work

We adopted a module-oriented collaboration strategy: each member took long-term ownership of a subsystem and the team merged changes continuously during development.

Member	Main Responsibilities	Key Files
Tong Wu (12412460) Role A: DHCP & Bonus DHCP	DHCP basic workflow, lease management, duplicate allocation prevention, DHCP option handling, and DHCP-related testing	<code>dhcp.py</code> , <code>tests/dhcp_test/</code> , bonus DHCP scripts
Zhikai Wang (12410615) Role B: Topology & Switching	Topology event handling, ARP processing, shortest-path computation, forwarding rule installation, dynamic topology updates, and switching complex case design	<code>controller.py</code> , <code>ofctl_utilis.py</code> , <code>tests/switching_test/</code> , complex topology scripts
Yunzang Li (12410922)		

Member	Main Responsibilities	Key Files
Role C: Firewall & Report/Experiment	Firewall rule loading and installation, experiment bonuses, report organization, figures, screenshots, and demonstration script support	firewall.py, firewall_rule.json, tests/firewall_test/, report/ and experiment scripts

5 Testing Methodology

We used Mininet-based scripts to validate both baseline functionality and complex dynamic scenarios. The main required tests are grouped into three categories.

5.1 DHCP Tests

- Basic DHCP allocation with default configuration.
- Custom DHCP configuration with modified `start_ip`, `end_ip`, and `netmask`.
- Address pool exhaustion where the number of hosts exceeds the pool size.

```
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
Sending DHCP request dhclient -v h1-eth0
Sending DHCP request dhclient -v h2-eth0
*** Starting CLI:
mininet> h1 ifconfig h1-eth0
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.2 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 00:00:00:00:00:01 txqueuelen 1000 (Ethernet)
    RX packets 5 bytes 1068 (1.0 KB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 6 bytes 1268 (1.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h2 ifconfig h2-eth0
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.3 netmask 255.255.255.0 broadcast 192.168.1.255
    ether 00:00:00:00:00:02 txqueuelen 1000 (Ethernet)
    RX packets 5 bytes 1068 (1.0 KB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 6 bytes 1268 (1.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h1 ping -c 2 192.168.1.3
PING 192.168.1.3 (192.168.1.3) 56(84) bytes of data.
64 bytes from 192.168.1.3: icmp_seq=1 ttl=64 time=4.06 ms
64 bytes from 192.168.1.3: icmp_seq=2 ttl=64 time=0.077 ms

--- 192.168.1.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1010ms
rtt min/avg/max/mdev = 0.077/2.070/4.064/1.993 ms
mininet>
```

Figure 1: Basic DHCP validation showing successful IP assignment and connectivity.

```

*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
h1: Sending DHCP request on eth0
h2: Sending DHCP request on eth0
h3: Sending DHCP request on eth0

===== Assigned Addresses =====
h1 -> 10.10.0.10
h2 -> 10.10.0.11
h3 -> 10.10.0.12

===== Connectivity Check h1 -> h2 =====
PING 10.10.0.11 (10.10.0.11) 56(84) bytes of data.

--- 10.10.0.11 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1028ms

*** Stopping 1 controllers
c0
*** Stopping 3 links
...
*** Stopping 1 switches
s1
*** Stopping 3 hosts
h1 h2 h3
*** Done

[PASS] DHCP custom configuration test passed

```

Figure 2: Custom DHCP configuration test with modified address pool and subnet mask.

```

s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1) (h5, s1)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 1 switches
s1 ...
h1: Sending DHCP request on eth0
h2: Sending DHCP request on eth0
h3: Sending DHCP request on eth0
h4: Sending DHCP request on eth0
h5: Sending DHCP request on eth0

===== Allocation Results =====
h1 -> 10.10.0.10
h2 -> None
h3 -> 10.10.0.12
h4 -> 10.10.0.11
h5 -> None

Expected: first 3 hosts receive IPs, remaining hosts do not.
Allocated hosts: h1, h3, h4
Unallocated hosts: h2, h5
*** Stopping 1 controllers
c0
*** Stopping 5 links
.....
*** Stopping 1 switches
s1
*** Stopping 5 hosts
h1 h2 h3 h4 h5
*** Done

[PASS] DHCP pool exhaustion test passed

```

Figure 3: Pool exhaustion test where only the first hosts receive addresses.

5.2 Shortest-Path Switching Tests

- Basic triangle topology with full host reachability.

- Complex topology with more than 6 hosts, more than 6 switches, and more than 10 edges.
- Dynamic topology changes including link down/up, switch stop/start, and port disable/en-able.

```

bin/python tests/switching_test/test_network.py
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (s1, s2) (s2, s3) (s3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)

```

Figure 4: Basic switching result in the triangle topology.

5.3 Firewall Tests

- Basic firewall policy enforcement in a single-switch topology.
- Complex multi-switch firewall case reusing the switching complex topology.
- Verification that previously reachable hosts become unreachable when firewall rules are installed, while unrelated traffic remains allowed.

```

bin/python tests/firewall_test/test_network.py
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
PING 192.168.117.3 (192.168.117.3) 56(84) bytes of data.

--- 192.168.117.3 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1040ms

PING 192.168.117.4 (192.168.117.4) 56(84) bytes of data.
64 bytes from 192.168.117.4: icmp_seq=1 ttl=64 time=3.34 ms
64 bytes from 192.168.117.4: icmp_seq=2 ttl=64 time=0.107 ms

--- 192.168.117.4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1026ms
rtt min/avg/max/mdev = 0.107/1.725/3.344/1.618 ms

curl: (28) Failed to connect to 192.168.117.3 port 80 after 2000 ms: Timeout was reached
HTTP_CODE=000

HTTP_CODE=200

```

Figure 5: Basic firewall validation results.

```

curl: (28) Failed to connect to 192.168.117.3 port 80 after 2010 ms: Timeout was reached
HTTP_CODE=000
EXIT=28

[PASS] h1 -> h2 TCP/8080 is allowed
HTTP_CODE=200
EXIT=0

[PASS] h3 -> h4 ICMP remains reachable
PING 192.168.117.5 (192.168.117.5) 56(84) bytes of data.
64 bytes from 192.168.117.5: icmp_seq=1 ttl=64 time=0.611 ms

--- 192.168.117.5 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.611/0.611/0.611/0.000 ms

[PASS] h6 -> h4 TCP/8081 remains reachable
HTTP_CODE=200
EXIT=0

[PASS] h7 -> h5 ICMP remains reachable
PING 192.168.117.6 (192.168.117.6) 56(84) bytes of data.
64 bytes from 192.168.117.6: icmp_seq=1 ttl=64 time=4.74 ms

--- 192.168.117.6 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.736/4.736/4.736/0.000 ms

[PASS] Reverse h2 -> h1 TCP/8081 is allowed
HTTP_CODE=200
EXIT=0

===== Summary: 7/7 checks passed =====

===== Firewall Flow Entries =====

```

Figure 6: Complex firewall validation results.

6 Complex Test Case

The project instructions explicitly require a complex topology with more than 6 hosts, more than 6 switches, and more than 10 edges. We designed such a topology and used it in both the

switching and firewall demonstrations.

6.1 Topology Summary

- Hosts: 7
- Switches: 7
- Inter-switch edges: 12

The complete Mermaid version of the topology and the host-to-host shortest path table are documented in `docs/complex-case-topology.md`.

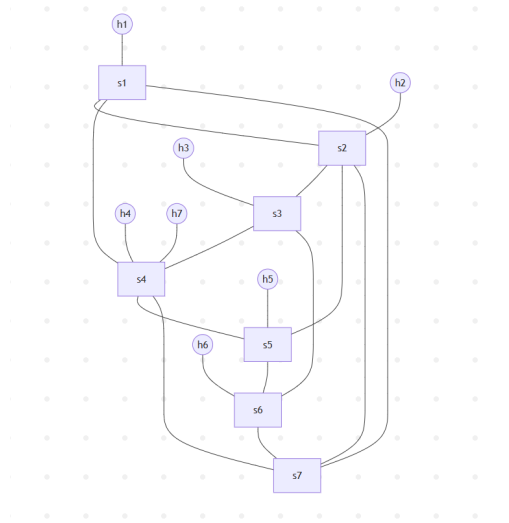


Figure 7: Complex topology used in the extended demonstration.

6.2 Dynamic Operations

The complex case supports dynamic operations from the Mininet CLI:

- link down/up,
- switch stop/start,
- port down/up through `ovs-ofctl mod-port`.

These operations are used to demonstrate that the controller can adapt its forwarding rules when the topology changes.

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)
mininet> link s2 s5 down
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)
mininet> link s2 s5 up
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7
h2 -> h1 h3 h4 h5 h6 h7
h3 -> h1 h2 h4 h5 h6 h7
h4 -> h1 h2 h3 h5 h6 h7
h5 -> h1 h2 h3 h4 h6 h7
h6 -> h1 h2 h3 h4 h5 h7
h7 -> h1 h2 h3 h4 h5 h6
*** Results: 0% dropped (42/42 received)

```

Figure 8: Connectivity validation during dynamic topology change operations in the complex case.

7 Bonus Results and Analysis

7.1 DHCP Bonus

The DHCP bonus extends the baseline allocator into a more complete DHCP service. On the server side, four state tables are maintained for pending offers, confirmed leases, lease expiration timestamps, and declined addresses that must not be reassigned. These states are sufficient to support lease recycling, renewal, and conflict handling in a structured way.

7.1.1 DHCP State Machine

The implementation follows the RFC 2131 client lifecycle rather than treating DHCP as a single DORA exchange. In particular, the server distinguishes the ordinary **DISCOVER** → **OFFER** → **REQUEST** → **ACK** path from **INIT-REBOOT**, **RENEWING**, and **REBINDING** requests, and it also supports explicit **RELEASE** and **DECLINE** handling. The result is a lease manager rather than a one-time address allocator.

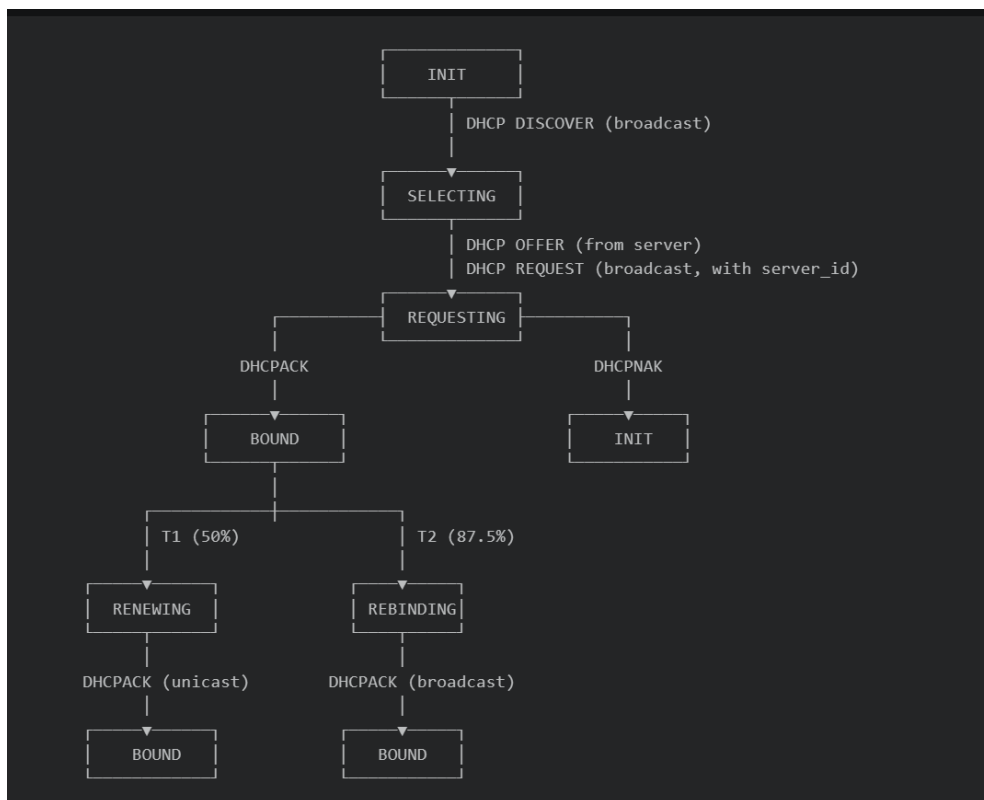


Figure 9: RFC 2131 client-side DHCP state transitions referenced by the lease-management logic.

This state machine is reflected directly in `dhcp.py`. The **REQUEST** handler does not treat all **REQUEST** messages as identical. Instead, it classifies them using `ciaddr`, `Option 50`, and `Option 54`, then applies the corresponding RFC logic.

```
if ciaddr == '0.0.0.0' and requested_ip and server_id_opt:
    # SELECTING
    ...

if ciaddr == '0.0.0.0' and requested_ip and not server_id_opt:
    # INIT-REBOOT
    ...

if ciaddr and ciaddr != '0.0.0.0':
```

```
# RENEWING / REBINDING
...
```

7.1.2 Lease Expiration

Lease expiration determines when an address can be returned to the pool without manual intervention. The server stores an expiration timestamp for each confirmed lease and reclaims that lease once the current time exceeds the recorded deadline.

```
@classmethod
def _is_expired(cls, ip: str) -> bool:
    expiry = cls.lease_expiry.get(ip)
    return expiry is not None and time.time() > expiry

@classmethod
def _reclaim_expired(cls):
    now = time.time()

    expired_macs = [
        mac for mac, ip in cls.ip_pool.items()
        if cls._is_expired(ip)
    ]
    for mac in expired_macs:
        ip = cls.ip_pool.pop(mac)
        cls.lease_expiry.pop(ip, None)
```

The natural-expiration test is used as the primary demonstration for this part. After the configured lease interval elapses, the original lease is reclaimed on the server side and the client can request a fresh lease again. This directly verifies expiration and reuse.

```
Step 1: Request initial DHCP lease
h1: Sending DHCP request on eth0
Initial IP: 192.168.1.2
Lease start time: 00:39:28
Lease will expire at: 00:39:58

Step 2: Stopping dhclient to prevent auto-renewal
This is critical - dhclient would auto-renew at T1 (50%% of lease time)
dhclient stopped

Step 3: Waiting for lease to expire (30 seconds)...
Remaining: 5 seconds
Lease has expired on server side!

Step 4: Verify lease expiry by requesting new DHCP lease
Note: Client IP may still be in kernel, but server lease has expired
Clearing old IP configuration...
Current IP after flush: None

Step 5: Requesting new DHCP lease...
h1: Sending DHCP request on eth0
New IP after expiry: 192.168.1.2
[PASSED]: Client successfully obtained new lease after expiry
```

Figure 10: Lease expiration test: the original lease is allowed to expire, the client state is cleared, and a new lease is requested successfully.

7.1.3 Duplicate-IP Prevention

To avoid duplicate allocation, the allocator performs reservation atomically. Before returning a new address, it first reclaims expired resources, then checks both the confirmed lease table and the pending-offer table, and immediately records the reservation in `pending_offers`. This prevents two nearby DISCOVER requests from observing the same address as free.

```
@classmethod
def _allocate_ip(cls, client_mac: str) -> Optional[str]:
    cls._reclaim_expired()

    if client_mac in cls.ip_pool:
        return cls.ip_pool[client_mac]
    if client_mac in cls.pending_offers:
        return cls.pending_offers[client_mac]["ip"]

    in_use = cls._in_use_ips()
    for ip in cls._build_ip_range():
        if ip not in in_use and ip not in cls.bad_ips:
            cls.pending_offers[client_mac] = {
                "ip": ip,
                "timestamp": time.time(),
            }
            return ip
    return None
```

The concurrent-allocation tests were useful during development, but the screenshots are less informative than the allocator logic itself. For that reason, the report keeps the mechanism and omits the redundant test images here.

7.1.4 OFFER Timeout Reclaim

The server also cleans up stale `pending_offers`. If a client receives an OFFER but never confirms it with REQUEST, that reservation is released after `offer_timeout` seconds and the address returns to the reusable pool.

```
expired_offer_macs = [
    mac for mac, info in cls.pending_offers.items()
    if now - info["timestamp"] > cls.offer_timeout
]
for mac in expired_offer_macs:
    info = cls.pending_offers.pop(mac)
```

```

Step 1: h1 sends DISCOVER (only DISCOVER, no REQUEST)
h1: Sending DHCP DISCOVER on eth0
DISCOVER sent at: 07:52:32
OFFER will expire at: 07:52:42

Step 2: Waiting 13 seconds for OFFER to timeout...
Remaining: 1 seconds
OFFER should have expired now!

Step 3: h1 re-sends DISCOVER
h1: Sending DHCP DISCOVER on eth0
h1 re-sent DISCOVER, pending offer should have been reclaimed

Step 4: h1 completes DORA
h1: Sending DHCP request on eth0
h1 IP: 192.168.1.2
[PASSED]: h1 obtained IP after pending OFFER expired

```

Figure 11: DHCP OFFER-timeout reclaim test result.

7.1.5 Lease Renewal

For RENEWING and REBINDING, the client already owns an address, so the server validates that the requested `ciaddr` still belongs to the same MAC address and then extends the lease deadline. This keeps renewal tied to the correct lease record.

```

Step 1: Get initial DHCP lease
h1: Sending DHCP request on eth0
Initial IP: 192.168.1.2
Original lease expiry time: 07:58:40

Step 2: Waiting 18 seconds for T1 (auto-renewal time)...
IP at T1 time: 192.168.1.2

Step 3: Wait until original expiry time...
Waiting 11 seconds to reach original expiry...

Step 4: Verify IP is still valid at original expiry time
IP at original expiry: 192.168.1.2
[PASSED]: IP still valid at original expiry (renewal successful)

```

Figure 12: Lease renewal test: the client renews the existing lease before expiration and keeps the assigned address.

```

if ciaddr and ciaddr != '0.0.0.0':
    existing_ip = cls.ip_pool.get(client_mac)
    if existing_ip != ciaddr:
        return cls._build_nak_pkt(client_mac, xid)
    if cls._is_expired(ciaddr):
        cls.ip_pool.pop(client_mac, None)
        cls.lease_expiry.pop(ciaddr, None)
        return cls._build_nak_pkt(client_mac, xid)

    cls.lease_expiry[ciaddr] = time.time() + cls.lease_time
    return cls._build_reply_pkt(
        client_mac, xid, ciaddr, dhcp.DHCP_ACK,
        unicast_ip=ciaddr
    )

```

7.1.6 DHCPNAK Handling

The server constructs RFC-compliant DHCPNAK packets for invalid or inconsistent requests. In particular, the reply uses `yiaddr = 0.0.0.0` and is broadcast, since the client may not currently hold a usable IP address.

```
@classmethod
def _build_nak_pkt(cls, client_mac: str, xid: int) -> packet.Packet:
    dhcp_pkt = dhcp.dhcp(
        op=dhcp.DHCP_BOOT_REPLY,
        chaddr=client_mac,
        siaddr=cls._gateway_ip,
        yiaddr='0.0.0.0',
        xid=xid,
        boot_file='',
        options=options,
    )
```

7.1.7 DECLINE and RELEASE

The implementation also supports active lease release and conflict reporting. RELEASE immediately returns the address to the pool, while DECLINE removes the address from the current state tables and inserts it into `bad_ips` so that it is not handed out again after a detected conflict.

```
@classmethod
def _handle_release(cls, pkt):
    ip = cls.ip_pool.pop(client_mac, None)
    if ip:
        cls.lease_expiry.pop(ip, None)
        cls.pending_offers.pop(client_mac, None)

@classmethod
def _handle_decline(cls, pkt):
    ...
    cls.bad_ips.add(declined_ip)
```

7.1.8 Broadcast and Unicast Reply Selection

The reply path distinguishes initialization from renewal. OFFER and initial ACK replies are broadcast when the client does not yet have an address, while renewal ACK replies may be unicast to the client's current IP.

```
use_broadcast = unicast_ip is None
dst_mac = 'ff:ff:ff:ff:ff:ff' if use_broadcast else client_mac
dst_ip = '255.255.255.255' if use_broadcast else unicast_ip
```

7.2 DNS Bonus

The DNS bonus demonstrates that the controller can also act as a simple DNS responder. This shows that the same SDN controller can host protocol logic beyond DHCP and forwarding.

The implementation is controller-side rather than host-side. A dedicated DNS helper is initialized from a static JSON record file and attached to the controller. When a switch joins, the controller installs a high-priority PacketIn flow for IPv4 UDP destination port 53, so DNS queries are sent to the controller instead of being handled by ordinary shortest-path forwarding.

In parallel, ARP requests for the virtual DNS IP 192.168.1.1 are answered by the controller, allowing hosts to send UDP packets to a controller-hosted virtual DNS server.

The DNS response path is implemented directly in `dns_server.py`. The controller extracts the UDP payload, parses the DNS question, looks up the queried name in the static table, and then builds a complete DNS response frame for PacketOut delivery back to the requesting host.

```
def install_packetin_flow(self, datapath, ofctl, ether_types, inet, vlan_id):
    actions = [
        datapath.ofproto_parser.OFPActionOutput(
            datapath.ofproto.OFPP_CONTROLLER,
            0xFFFF,
        )
    ]
    ofctl.set_flow(
        cookie=self.COOKIE,
        priority=self.PACKETIN_PRIORITY,
        dl_type=ether_types.ETH_TYPE_IP,
        dl_vlan=vlan_id,
        nw_proto=inet.IPPROTO_UDP,
        tp_dst=self.DNS_PORT,
        actions=actions,
    )
```

```
def build_response(self, query):
    query_id, flags, qdcount, _, _, _ = struct.unpack("!HHHHHH", query[:12])
    question_end, qname = self._read_qname(query, 12)
    question = query[12:question_end + 4]
    qtype, qclass = struct.unpack("!HH", query[question_end:question_end + 4])
    answer_ip = self.records.get(self._normalize_name(qname))

    answer = b""
    rcode = 0
    if qtype == 1 and qclass == 1 and answer_ip:
        answer = self._build_a_answer(answer_ip)
    elif qtype == 1 and qclass == 1:
        rcode = 3

    response_flags = 0x8000 | 0x0400 | (flags & 0x0100) | rcode
    header = struct.pack("!HHHHHH", query_id, response_flags, 1, 1 if answer else 0, 0, 0)
    return header + question + answer
```

The test topology is a simple `h1-s1-h2` network. Three DNS cases are checked: `web.cs305.local` resolving to 192.168.1.3, `h1.cs305.local` resolving to 192.168.1.2, and an unknown name returning NXDOMAIN. The script also runs `pingAll()` to verify that normal forwarding behavior is not broken by the DNS extension, and it prints the switch flow table to confirm that the UDP/53 PacketIn rule is present.


```

h2 -> h1
*** Results: 0% dropped (2/2 received)

===== DNS bonus test results =====

[PASS] web.cs305.local resolves to 192.168.1.3
DNS_FLAGS=0x8500 ANSWERS=1 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] h1.cs305.local resolves to 192.168.1.2
DNS_FLAGS=0x8500 ANSWERS=1 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] missing.cs305.local returns NXDOMAIN
DNS_FLAGS=0x8503 ANSWERS=0 FROM=192.168.1.1
DNS_TEST_PASS
EXIT=0

[PASS] normal shortest-path forwarding still works
packet loss=0.0%

===== DNS PacketIn flow on s1 =====
cookie=0x305d, duration=8.339s, table=0, n_packets=1, n_bytes=75, idle_age=0, priority=2000,udp,tp_dst=53 actions=CONTROLLER:65535
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s1
*** Stopping 2 hosts
h1 h2
*** Done

===== Summary: 4/4 checks passed =====
(cs305) w1369@LAPTOP-UPKABV41:~/computer_network/make_proj/CS305-26Spring-Project$
[work] 0:[tmux] 1:node 2:python3.8- 3:bash* 4:bash "LAPTOP-UPKABV41" 10:59 04-Jun-26

```

Figure 13: DNS bonus test results.

7.3 TCP Congestion Bonus

The TCP congestion experiment compares algorithm behavior under the same bottleneck. Our results show that the different algorithms produce different throughput values, while fairness remains close to 1 in the shared-flow scenario.

This bonus is implemented as a Mininet experiment rather than a controller feature. The topology contains four hosts and two switches, with a single bottleneck link between `s1` and `s2`. Access links are configured as 100 Mbps, 1 ms, while the bottleneck link is configured as 10 Mbps, 30 ms, 0.5% loss, queue size 100. The loss is introduced directly by `TCLink` to create a controlled environment for comparing the recovery behavior of different TCP congestion-control algorithms.

The script performs two experiments. The first is a single-flow test, where `h1 -> h2` runs once with Reno and once with CUBIC. The second is a fairness test, where `h1 -> h2` uses Reno and `h3 -> h4` uses CUBIC simultaneously while sharing the same bottleneck. Throughput is parsed from `iperf` output, and fairness is computed with Jain's fairness index.

```

def set_tcp_algorithm(host, algorithm):
    host.cmd("modprobe tcp_%s 2>/dev/null || true" % algorithm)
    output = host.cmd(
        "sysctl -w net.ipv4.tcp_congestion_control=%s 2>&1" % algorithm
    )
    current = host.cmd("sysctl -n net.ipv4.tcp_congestion_control 2>/dev/null").strip()
    if current != algorithm:
        return False
    return True

```

```

def jain_fairness(x1, x2):
    if x1 is None or x2 is None:
        return None
    denominator = 2 * (x1 ** 2 + x2 ** 2)
    if denominator == 0:
        return None
    return (x1 + x2) ** 2 / denominator

```

The expected qualitative result is that CUBIC usually achieves higher average throughput than Reno in the single-flow case because it recovers the congestion window more aggressively after loss. At the same time, the fairness test is expected to remain close to 1, meaning that the two flows still share the bottleneck reasonably evenly even though their congestion-control behavior differs. In our repeated runs, this is exactly the trend that appears: CUBIC shows better average single-flow throughput, while fairness stays high overall.

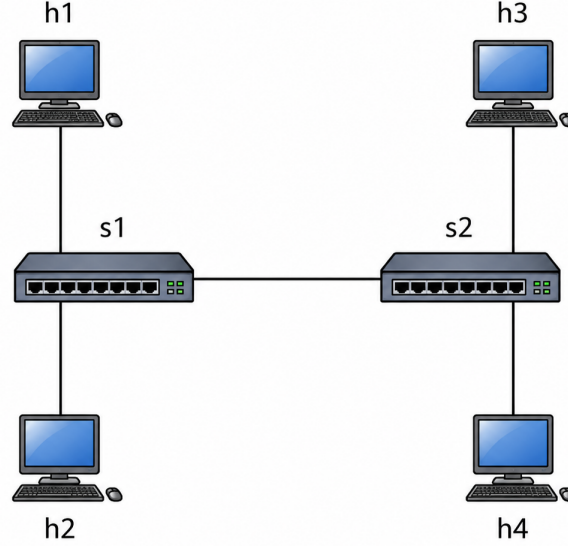


Figure 14: Mininet topology used in the TCP congestion and bufferbloat bonus experiments.

```

===== Result Summary =====

Single flow throughput:
algorithm    loss(%)  throughput(Mbps)
reno         0.5      4.05
cubic        0.5      4.43

Fairness test:
reno(Mbps)   cubic(Mbps)   fairness
3.9          2.95         0.9811291165708309

Conclusion data:
single_reno(Mbps)  single_cubic(Mbps)  fair_reno(Mbps)  fair_cubic(Mbps)  fairness
4.05              4.43              3.9              2.95              0.9811291165708309

```

Figure 15: TCP congestion experiment results for Reno, CUBIC, and the fairness test.

7.4 Bufferbloat Bonus

The bufferbloat experiment shows the classic tradeoff between throughput and delay. With a very large queue, packet loss can decrease and throughput can improve, but the busy RTT becomes much larger, which clearly demonstrates queueing delay inflation.

This experiment reuses the same two-switch structure but changes the bottleneck configuration. The access links remain 100 Mbps, 1 ms, while the s1-s2 bottleneck is set to 1 Mbps, 20 ms. The only parameter varied between experiments is the bottleneck queue size: a small queue of 20 packets and a large queue of 1000 packets.

Two traffic types are run simultaneously. A long-lived TCP flow from h1 to h3 is generated with `iperf`, and an interactive probe flow from h2 to h4 is generated with `ping`. The script

first measures an idle RTT baseline, then starts the background ping process, launches the TCP bulk transfer, and finally parses both the `iperf` output and the ping statistics.

```
def parse_ping(output):
    loss_match = re.search(r"(\d+(?:\.\d+)?)% packet loss", output)
    rtt_match = re.search(
        r"(?:rtt|round-trip) min/avg/max/(?:mdev|stddev) = "
        r"[\d.]+/([\d.]+)/[\d.]+/[\d.]+",
        output,
    )
    loss = float(loss_match.group(1)) if loss_match else None
    avg_rtt = float(rtt_match.group(1)) if rtt_match else None
    return loss, avg_rtt
```

```
idle_ping = ping(h2, h4.IP())
idle_loss, idle_rtt = parse_ping(idle_ping)
h3.cmd("iperf -s -p 5001 >/tmp/bonus5_iperf_server.log 2>&1 &")
h2.cmd("ping -c 30 -i 0.5 -W 2 %s >/tmp/bonus5_ping.log 2>&1 &" % h4.IP())
iperf_output = h1.cmd("iperf -c %s -p 5001 -t %s" % (h3.IP(), duration))
busy_ping = h2.cmd("cat /tmp/bonus5_ping.log")
busy_loss, busy_rtt = parse_ping(busy_ping)
throughput = parse_iperf(iperf_output)
```

The expected outcome is a classic bufferbloat pattern. The large queue tends to reduce loss and may improve TCP throughput because packets are buffered instead of being dropped early. However, that same queue causes small interactive packets to wait behind the TCP bulk transfer, so the busy RTT rises sharply. In our measured results, the large queue indeed produced higher throughput and lower loss, but at the cost of dramatically larger RTT under load, which is the core symptom of bufferbloat.

```
*** Starting controller

*** Starting 2 switches
s1 s2 ...(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
r2q change.
(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
e.
(1.00Mbit 20ms delay) (100.00Mbit 1ms delay) *** Error: Warning: sch_htb: q
ig. Consider r2q change.
(100.00Mbit 1ms delay) *** Error: Warning: sch_htb: quantum of class 50001
e.
(1.00Mbit 20ms delay)

===== Queue size: 1000 packets =====
*** Stopping 0 controllers

*** Stopping 5 links
....
*** Stopping 2 switches
s1 s2
*** Stopping 4 hosts
h1 h2 h3 h4
*** Done

===== Summary =====
queue   throughput(Mbps)   idle_rtt(ms)   busy_rtt(ms)   loss(%)
20      0.954             50.961         327.488        13.3333
1000    1.3               51.227         1820.666       0.0
```

Figure 16: Bufferbloat experiment results comparing small and large bottleneck queues.

8 Conclusion

This project delivers a functional SDN controller that combines DHCP, shortest-path switching, and firewall enforcement in one os-ken application. The implementation is supported by Mininet-based testing, a complex topology for robustness demonstration, and bonus experiments that extend both protocol functionality and network measurement depth.

The final report should be read together with the test scripts and demo documents in the repository, especially the complex topology documentation and the defense guide.